

FROM STAKEHOLDER TASKS TO ARCHITECTURE VIEWS

A Knowledge- and Evidence-Grounded Multi-Agent Approach

Research idea and study plan



Internal discussion draft

August 2026

Executive Summary

This research investigates how large language model (LLM) agents can generate the right software architecture views for a specific stakeholder and task, rather than producing generic documentation or a fixed set of diagrams. The system receives a software repository and a natural-language description of the user, task, target, and goal; it then derives the relevant architectural questions, selects suitable views, plans the repository investigation, extracts grounded evidence, and generates validated stakeholder-task-oriented views.

RESEARCH NEED	PROPOSED APPROACH	EXPECTED OUTPUT
Architecture information must be selected according to who needs it and which task must be performed.	A multi-agent pipeline connects stakeholder needs, architectural knowledge, repository analysis planning, and evidence-grounded generation.	A validated set of architecture views with rationale, answered questions, evidence links, limitations, and confidence.

Core contribution

Natural-language stakeholder need + knowledge-grounded viewpoint selection + agentic evidence extraction and grounding + validated view generation.

Working Title

From Stakeholder Tasks to Architecture Views: A Knowledge- and Evidence-Grounded Multi-Agent Approach

Alternative title: Task2View: Agentic Generation of Stakeholder-Oriented Architecture Views from Software Repositories

Document Map

- Research problem, motivation, objective, and scope
- Overall multi-agent process and operational input/output summary
- Detailed specification of the eight phases, including illustrative inputs and outputs
- Architecture Evidence Model and final deliverables
- Scientific contribution, research questions, evaluation design, and risks
- Implementation roadmap and proposed paper structure

1. Research Idea and Motivation

1.1 Core Idea

The proposed approach automatically generates software architecture views tailored to a specific stakeholder and task. It does not ask the user to select a diagram type or formulate architectural questions. Instead, the multi-agent system derives what the stakeholder needs to know, determines which views can answer those needs, and generates only the views that are supported by evidence in the software repository.

```
Software Repository + Natural-Language Stakeholder/Task Description
                        ↓
Stakeholder-Task-Oriented Architecture Views
```

1.2 Motivation

Different stakeholders need different architectural information. A developer implementing a feature, a tester designing integration tests, a maintainer assessing change impact, a DevOps engineer analyzing deployment, and a security analyst examining trust boundaries do not need the same views or the same level of detail.

- Generic documentation can be technically plausible but irrelevant to the concrete task.
- Fixed templates may overproduce information while omitting task-critical concerns.
- Direct repository-to-diagram generation often skips explicit reasoning about stakeholder needs, view selection, and evidence sufficiency.
- Architecture views should therefore be selected and generated as answers to task-specific architectural questions.

1.3 Research Problem

Research problem

Which architecture views should be generated for a particular stakeholder and task, and how can the system ensure that those views are grounded in repository evidence?

1.4 Research Objective

The objective is to design and evaluate a multi-agent approach that transforms a repository and a natural-language stakeholder-task description into architecture views that are relevant, appropriately abstracted, traceable, evidence-grounded, and useful for the intended task.

1.5 Scope and Main Design Decision

All core architectural reasoning and repository interpretation activities are performed by LLM-based agents. The approach does not rely on deterministic architecture-recovery tools, static-analysis engines, call-graph generators, or predefined source-code parsers as the primary mechanism for extracting architecture information.

Agents may use basic repository-access operations—such as listing directories, searching symbols, opening files, and reading code fragments—but the identification of relevant evidence, interpretation of relationships, abstraction of architecture elements, and confidence assessment remain responsibilities of the LLM agents.

2. Inputs and Final Output

2.1 Input 1 — User Description

The user provides a free-text description containing the stakeholder role, task, target, intended goal, scope, constraints, and optional preferences.

Example

```
I am an integration tester. I need to create tests for the payment functionality and understand which components are involved, in which order they interact, which external services are used, and which failure paths should be tested.
```

2.2 Input 2 — Software Repository

The repository can be supplied as a local path, a GitHub URL, a cloned project, or an archive. Potentially relevant artifacts include source code, tests, configuration, dependency manifests, API specifications, container and deployment descriptors, database schemas, build scripts, CI/CD files, and existing documentation.

2.3 Final Output

The final deliverable is a set of validated Stakeholder-Task-Oriented Architecture Views. Each view is accompanied by its rationale, addressed architectural questions, evidence links, limitations, unanswered questions, and confidence information.

3. Overall Multi-Agent Process

Figure 1 summarizes the proposed sequential pipeline. Process boxes are executed by agents, while green shapes represent explicit intermediate artifacts. The Repository Analysis Plan is the key artifact separating investigation planning in Phase 4 from evidence extraction in Phase 5.

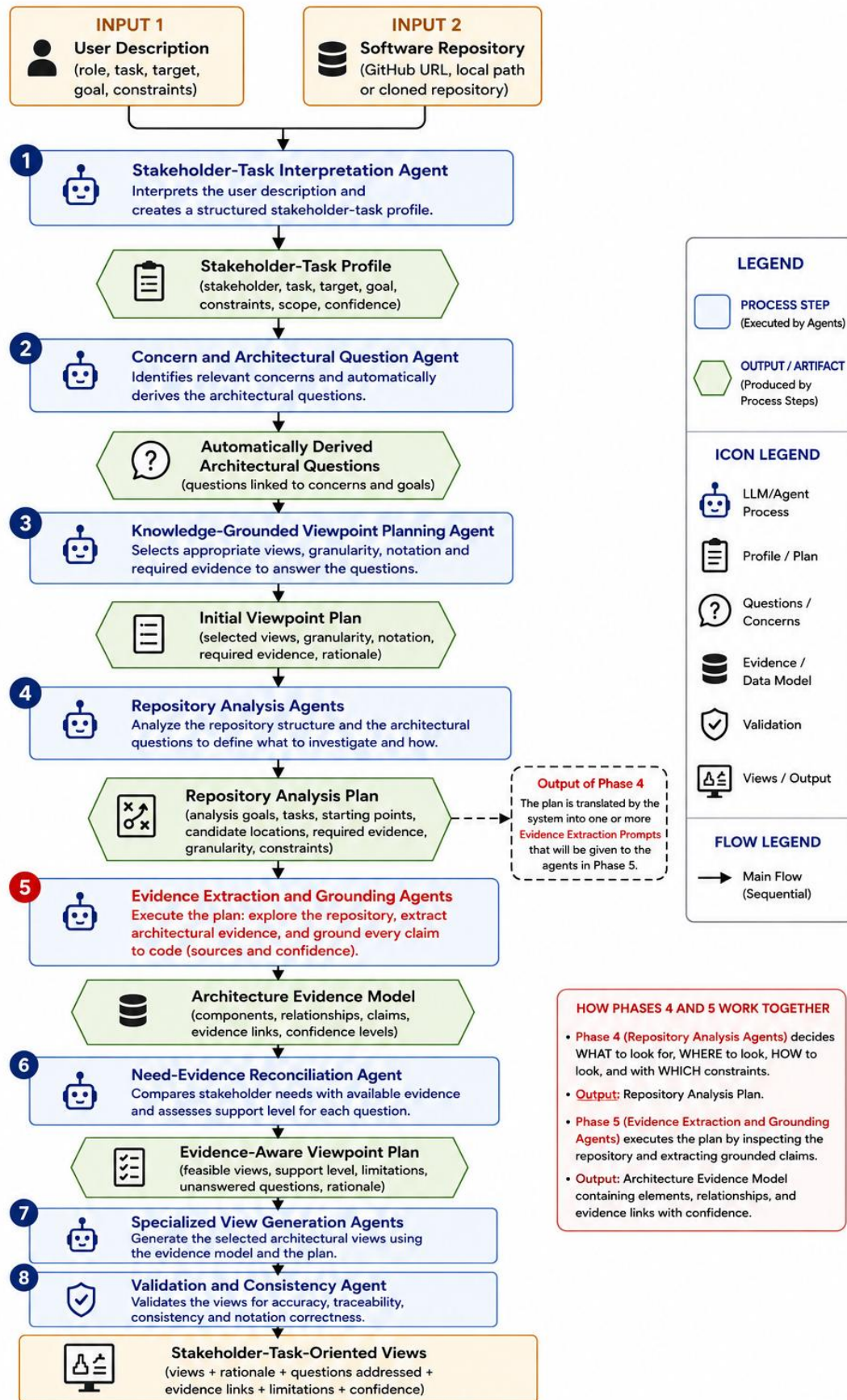


Figure 1. Overall multi-agent architecture and process.

4. Operational Summary: Inputs and Outputs by Phase

Phase	Agent / Component	Inputs	Outputs
1	Stakeholder-Task Interpretation Agent	User description	Stakeholder-Task Profile
2	Concern and Architectural Question Agent	Stakeholder-Task Profile	Architectural Concerns and Questions
3	Knowledge-Grounded Viewpoint Planning Agent	Profile, questions, architecture knowledge base	Initial Viewpoint Plan
4	Repository Analysis Agents	Repository, questions, Initial Viewpoint Plan	Repository Analysis Plan, including candidate locations, and extraction prompts
5	Evidence Extraction and Grounding Agents	Repository, Repository Analysis Plan, prompts	Grounded claims and Architecture Evidence Model
6	Need-Evidence Reconciliation Agent	Initial Viewpoint Plan, questions, evidence model	Evidence-Aware Viewpoint Plan
7	Specialized View Generation Agents	Evidence-aware plan and evidence model	Candidate Architecture Views
8	Validation and Consistency Agent	Candidate views, evidence model, questions, profile	Validated Stakeholder-Task-Oriented Views

5. Detailed Process Specification

Each phase below includes an operational input/output summary and a compact example showing how information is transformed across the pipeline.

Phase 1 — Stakeholder-Task Interpretation

Stakeholder-Task Interpretation Agent

The agent transforms the user's free-text description into a structured representation of who needs the information, which task must be performed, what part of the system is relevant, and which constraints should guide the analysis.

INPUTS	OUTPUT
• User description • Optional scope and preferences	• Stakeholder-Task Profile
Core operation: Extract stakeholder, task, target, goal, scope, constraints, desired detail, and confidence.	
ILLUSTRATIVE INPUT User description “I am an integration tester. I need to design tests for payment processing and understand the participating components, their interaction order, external services, and failure paths.”	ILLUSTRATIVE OUTPUT Stakeholder-Task Profile Stakeholder: Integration Tester Task: Design integration tests Target: Payment processing Goal: Recover components, interactions, external dependencies, and failure paths Confidence: 0.94

Illustrative output

```
{
  "stakeholder": "Integration Tester",
  "task": "Design integration tests",
  "target": "Payment functionality",
  "goal": "Understand components, interactions and failure paths",
  "scope": "Payment-related functionality",
  "confidence": 0.94
}
```

Phase 2 — Concern and Architectural Question Derivation

Concern and Architectural Question Agent

The agent derives the architectural concerns and questions that must be answered for the stakeholder to perform the task. These questions are generated automatically; they are not supplied by the user.

INPUTS	OUTPUT
- Stakeholder-Task Profile - General knowledge about stakeholder responsibilities and task needs	- Architectural Concerns - Automatically Derived Architectural Questions
Core operation: Translate the task into explicit information needs and prioritized questions.	
ILLUSTRATIVE INPUT Stakeholder-Task Profile Stakeholder: Integration Tester Task: Design integration tests Target: Payment processing Goal: Understand the runtime workflow and test-relevant failures	ILLUSTRATIVE OUTPUT Architectural concerns - Runtime interactions; external dependencies; data exchange; failure handling Derived questions - AQ1. Which components participate? - AQ2. In which order do they interact? - AQ3. Which external systems and failure paths are involved?

- Which components participate in payment processing?
- In which order do they interact?
- Which external systems are involved?
- Which data are exchanged?
- Which failure paths are implemented?

Phase 3 — Knowledge-Grounded Viewpoint Planning

Knowledge-Grounded Viewpoint Planning Agent

The agent uses an architectural knowledge base to select the view types, abstraction level, notation, and evidence required to answer the derived questions.

INPUTS	OUTPUT
- Stakeholder-Task Profile - Architectural concerns and questions - Architectural knowledge base	Initial Viewpoint Plan
Core operation: Map stakeholder/task/concern/question to view type, required evidence, notation, and granularity.	

ILLUSTRATIVE INPUT Questions and knowledge-base mappings • AQ1–AQ3 from Phase 2 • Runtime interaction → Sequence View • External dependencies → Component Interaction View Target granularity: component level	ILLUSTRATIVE OUTPUT Initial Viewpoint Plan V1: Component Interaction View for AQ1 and AQ3 V2: Sequence View for AQ2 and failure-related questions Required evidence: responsibilities, calls, interaction order, external invocations, and alternative paths
---	--

The architectural knowledge base should combine retrievable documents with structured mappings such as:

Stakeholder → Task → Concern → Architectural Question
→ View Type → Required Evidence → Recommended Granularity

The Initial Viewpoint Plan describes what would ideally be generated, but it does not yet guarantee that the repository contains enough evidence.

Phase 4 — Repository Analysis

Repository Analysis Agents

Phase 4 is the investigation-planning stage. Starting from the software repository, the Architectural Questions, and the Initial Viewpoint Plan, the agents perform a high-level repository reconnaissance to determine what must be searched and to identify the candidate files, packages, directories, and artifacts that should be inspected in depth. They then decompose the investigation into evidence-extraction tasks and define the constraints that the Phase 5 agents must respect.

INPUTS	OUTPUT
• Software repository • Stakeholder-Task Profile • Architectural Questions • Initial Viewpoint Plan • Required evidence and granularity	• Repository Analysis Plan, including candidate directories, files, packages, starting points, analysis tasks, and constraints • One or more Evidence Extraction Prompts
Core operation: Define what, where, and how to investigate; do not yet claim what the architecture actually contains.	
ILLUSTRATIVE INPUT Repository + Initial Viewpoint Plan Repository: payment-system Architectural question: Which components participate in the payment workflow and in which order? Selected view: Component-level Sequence View Required evidence: workflow entry points, internal interactions, external systems, interaction order, and failure paths	ILLUSTRATIVE OUTPUT Repository Analysis Plan AT1: identify workflow entry points Candidate locations: src/controllers/ and openapi.yaml AT2: recover internal interactions Candidate locations: src/services/ and src/repositories/ AT3: identify external systems and failure paths Candidate locations: src/clients/, application.yml, tests/integration/, and service implementations Constraints: component-level granularity; grounded relationships; observed/inferred distinction

Phase 4 practical activities

1. Perform repository reconnaissance: inspect directory structure, languages, manifests, tests, API descriptions, configuration, deployment artifacts, and likely entry points.
2. Translate each architectural question into concrete investigation goals.
3. Use the initial reconnaissance to identify candidate directories, files, packages, symbols, and artifact types where evidence is likely to be found; these candidate locations become part of the Repository Analysis Plan.

4. Decompose the investigation into smaller evidence-extraction tasks.
5. Define target granularity, evidence schema, terminology, and constraints.
6. Produce a structured Repository Analysis Plan that can be translated into prompts for Phase 5.

Repository Analysis Plan — simplified example

```
{
  "architectural_question": "Which components participate in payment processing and in which order?",
  "analysis_goal": "Recover the component-level payment workflow",
  "target_view": "Sequence View",
  "target_granularity": "Component level",
  "analysis_tasks": [
    {
      "goal": "Identify the workflow entry point",
      "candidate_locations": ["src/controllers/", "src/api/", "openapi.yaml"],
      "expected_evidence": ["endpoint", "initial component", "invoked operation"]
    },
    {
      "goal": "Recover internal component interactions",
      "candidate_locations": ["src/services/", "src/repositories/"],
      "expected_evidence": ["source", "target", "relationship", "order"]
    }
  ],
  "constraints": [
    "Maintain component-level granularity",
    "Ground every relationship",
    "Distinguish observed and inferred information"
  ]
}
```

Implementation note

Conceptually, the output is the Repository Analysis Plan. Operationally, the system translates the plan into one or more task-specific prompts for the Evidence Extraction and Grounding Agents.

Phase 5 — Evidence Extraction and Grounding

Evidence Extraction and Grounding Agents

Phase 5 executes the Repository Analysis Plan. The agents inspect the selected repository artifacts, follow relevant references, extract atomic architectural claims, and ground each claim in files, symbols, and code excerpts.

INPUTS	OUTPUT
• Software repository • Repository Analysis Plan • Evidence Extraction Prompts • Expected output schema and constraints	• Grounded Architectural Claims • Architecture Evidence Model
Core operation: Inspect, extract, classify, and ground what the repository actually supports.	
ILLUSTRATIVE INPUT Evidence-extraction task generated from the plan Goal: recover internal payment interactions Starting locations: src/services and src/repositories Return: source, target, relationship, order, file, symbol, excerpt, support type, confidence	ILLUSTRATIVE OUTPUT Grounded claim / AEM relation Claim: PaymentService invokes StripeClient Evidence: src/services/PaymentService.java — processPayment() Excerpt: stripeClient.charge(payment) Support: directly observed; Confidence: 0.97

Phase 5 practical activities

1. Open and interpret the candidate artifacts identified in the plan.
2. Follow imports, calls, interfaces, implementations, configuration entries, tests, API routes, and deployment references.
3. Inspect additional artifacts discovered during exploration; candidate locations are starting points, not a closed boundary.
4. Produce small, verifiable claims before creating any complete architecture view.
5. Link every claim to repository evidence: file, symbol or method, relevant excerpt, support type, and confidence.
6. Classify evidence as directly observed, inferred from multiple artifacts, weakly supported, or unsupported.
7. Consolidate supported elements, relationships, behavior, provenance, and confidence into the Architecture Evidence Model.

Illustrative extraction prompt generated from Phase 4

You are an Evidence Extraction and Grounding Agent.

Task: recover the component-level payment workflow.

Question: Which components participate in payment processing and in which order?

Start from controllers, services, repositories, clients, integration tests, API specifications, and configuration files. For each recovered interaction, report the source and target components, relationship type, order when supported, file, symbol, code excerpt, support type, and confidence. Maintain component-level granularity and do not introduce unsupported elements.

Grounded claim — simplified example

```
{
  "claim": "PaymentService invokes StripeClient",
  "source_element": "PaymentService",
  "target_element": "StripeClient",
  "relationship": "invokes",
  "evidence": [{
```

```

    "file": "src/services/PaymentService.java",
    "symbol": "processPayment",
    "excerpt": "stripeClient.charge(payment) "
  }},
  "support_type": "directly_observed",
  "confidence": 0.97
}

```

Why Phases 4 and 5 are separate

PHASE 4 — PLAN	PHASE 5 — EXECUTE
Defines what to look for, where to start, how to split the work, which evidence is required, and which constraints apply. Output: Repository Analysis Plan.	Reads the repository, follows references, extracts and verifies claims, attaches provenance, and builds the Architecture Evidence Model.

Phase 6 — Need-Evidence Reconciliation

Need-Evidence Reconciliation Agent

The agent compares the information that would be useful for the stakeholder with the information that is actually supported by the repository. It prevents the system from generating a view merely because the view was initially desirable.

INPUTS	OUTPUT
- Stakeholder-Task Profile - Architectural Questions - Initial Viewpoint Plan - Architecture Evidence Model	Evidence-Aware Viewpoint Plan
Core operation: Classify each question and planned view as fully, partially, weakly, or not supported; adapt scope and granularity.	
ILLUSTRATIVE INPUT Planned need vs. recovered evidence - The Sequence View requires ordered calls and failure paths. - The evidence model supports the main call chain but contains no reliable evidence for asynchronous callbacks and only partial timeout handling.	ILLUSTRATIVE OUTPUT Evidence-Aware Viewpoint Plan Sequence View: feasible with limitations Main workflow: fully supported Failure paths: partially supported Asynchronous callbacks: unsupported and explicitly omitted

- Confirm a view when sufficient evidence exists.
- Simplify a view or change its granularity when evidence is partial.
- Attach explicit limitations and unanswered questions.
- Do not generate unsupported views as factual representations.

Phase 7 — Specialized View Generation

Specialized View Generation Agents

Specialized agents generate the selected views using the Evidence-Aware Viewpoint Plan and Architecture Evidence Model. Possible agents include structural, behavioral, deployment, data, security, and dependency/change-impact view generators.

INPUTS	OUTPUT
--------	--------

• Evidence-Aware Viewpoint Plan • Architecture Evidence Model • Architectural Questions • Notation and granularity rules	• Candidate Architecture Views
Core operation: Render only elements and relationships that are supported by the evidence model.	
ILLUSTRATIVE INPUT Evidence-aware plan + Architecture Evidence Model • Generate a component-level Sequence View using only supported relations: • Payment API → Payment Service → Payment Repository • Payment Service → External Payment Provider	ILLUSTRATIVE OUTPUT Candidate Architecture View • A sequence diagram containing the supported participants and ordered messages Rationale: supports integration-test design Addressed questions: AQ1–AQ3 Limitation: asynchronous callback behavior is not represented

Each candidate view includes its purpose, rationale, addressed questions, evidence links, limitations, and confidence.

Phase 8 — Validation and Consistency

Validation and Consistency Agent

The validation agent checks repository faithfulness, question coverage, task alignment, cross-view consistency, and notation correctness. Candidate views can be corrected or regenerated internally, while the high-level process remains sequential.

INPUTS	OUTPUT
• Candidate Architecture Views • Architecture Evidence Model • Evidence-Aware Viewpoint Plan • Architectural Questions • Stakeholder-Task Profile	• Validated Stakeholder-Task-Oriented Views
Core operation: Verify evidence support, task relevance, consistency, readability, and syntax before delivery.	
ILLUSTRATIVE INPUT Candidate view + evidence + validation criteria • Candidate Sequence View, Architecture Evidence Model, stakeholder task, assigned questions, notation rules, and known limitations	ILLUSTRATIVE OUTPUT Validated Stakeholder-Task-Oriented View Status: pass with minor corrections • Removed an unsupported timeout branch • Normalized component names across views • Delivered with evidence links, confidence, addressed questions, and explicit limitations

- Every represented element and relationship must be supported by evidence.
- Inferred information must be marked explicitly.
- Names, boundaries, and relationships must be consistent across views.
- The view must answer its assigned questions at an appropriate abstraction level.
- Unanswered questions and limitations must remain visible to the user.

6. Architecture Evidence Model

The Architecture Evidence Model is the central intermediate representation shared by the agents. It separates repository interpretation from view rendering and allows the same grounded evidence to support multiple views.

- Architecture elements: components, services, packages, classes, APIs, data stores, external systems, deployment units, and runtime nodes.
- Architecture relationships: dependencies, calls, data flows, interaction order, deployment mappings, and trust-boundary crossings.
- Semantic information: responsibilities, boundaries, interfaces, and architectural roles.
- Grounding information: files, symbols, code excerpts, support type, confidence, and question coverage.

Evidence rule

No element or relationship should appear in a generated view unless it is represented in the Architecture Evidence Model and linked to repository evidence.

7. Final Output Package

- Validated architecture views selected for the specific stakeholder and task.
- A rationale explaining why each view was selected and how it supports the task.
- The architectural questions addressed by each view.
- Traceability links from elements and relationships to repository artifacts.
- Support type and confidence for relevant claims.
- Explicit limitations, unavailable evidence, and unanswered questions.

8. Scientific Contribution and Positioning

8.1 Main Contribution

The proposed contribution is the integration of three capabilities that are often treated separately: natural-language need interpretation, knowledge-grounded viewpoint selection, and evidence-grounded architecture-view generation.

Contribution statement

Natural-language need + knowledge-grounded view selection + agentic repository investigation planning + evidence-grounded view generation.

8.2 Expected Novelty

- The process starts from a stakeholder-task description rather than a predefined documentation template.
- Architectural questions are derived automatically as the bridge between the task and the view selection.
- Viewpoint planning occurs before deep repository analysis and defines the required evidence and granularity.
- The Repository Analysis Plan explicitly separates investigation planning from evidence extraction.
- Evidence is extracted by dedicated LLM agents and grounded at claim level in repository artifacts.
- Need-evidence reconciliation prevents unsupported or overconfident view generation.
- Validation considers not only syntax and consistency, but also task relevance and question coverage.

8.3 Concise Positioning Statement

The approach does not ask an LLM simply to inspect a repository and generate an architecture diagram. It first determines who needs the information, which task must be performed, which questions must be answered, which views are appropriate, how the repository should be investigated, which evidence is available, and which views can be generated faithfully.

```
Stakeholder Need → Architectural Questions → Viewpoint Selection
→ Repository Investigation Plan → Grounded Architectural Evidence
→ Evidence-Aware View Generation → Validated Views
```

9. Research Questions

RQ1 — Stakeholder-Task Interpretation and View Planning

To what extent can the proposed approach derive appropriate architectural questions and select architecture views that are relevant to a stakeholder’s task?

Evaluation focus: correctness of task interpretation, relevance of questions, appropriateness of selected views, and granularity.

RQ2 — Task Support

To what extent do the generated architecture views support stakeholders in performing their intended tasks?

Evaluation focus: task completion, decision support, information sufficiency, reduction of repository-inspection effort, and perceived usefulness.

RQ3 — Repository Faithfulness

To what extent are the generated architecture views faithful to the analyzed software repository?

Evaluation focus: correctness of elements and relationships, evidence traceability, unsupported claims, completeness with respect to available evidence, and cross-view consistency.

Optional RQ4 — Efficiency and Cost

What are the time, token cost, and human correction effort required by the proposed multi-agent process?

10. Evaluation Plan

10.1 Evaluation Dimensions

Stakeholder-Task Adequacy

- Correctness of stakeholder and task interpretation
- Relevance of derived concerns and questions
- Appropriateness of selected view types and granularity

Task Utility

- Task completion rate and completion time
- Correctness of task answers or decisions
- Perceived usefulness and information sufficiency
- Cognitive effort and need for direct source-code inspection

Repository Faithfulness

- Element and relationship precision
- Behavioral-order correctness
- Evidence correctness and traceability
- Unsupported-element and unsupported-relationship rates
- Question coverage, cross-view consistency, and diagram syntax

10.2 Subjects, Tasks, and Systems

Potential participants include developers, testers, maintainers, architects, DevOps engineers, and security engineers. Representative tasks may include feature understanding, integration-test design, change-impact analysis, external-dependency identification, deployment understanding, trust-boundary analysis, and subsystem onboarding.

Repositories should vary in size, language, domain, architectural style, documentation quality, test availability, and deployment artifacts.

10.3 Comparison and Ablation Conditions

- Repository inspection without generated architecture views
- Generic LLM-generated architecture documentation
- Fixed-template documentation generation
- Direct repository-to-view generation
- Complete proposed multi-agent approach

Ablations can remove stakeholder-task interpretation, architectural knowledge, the Repository Analysis Plan, claim grounding, need-evidence reconciliation, or validation to isolate the contribution of each mechanism.

11. One-Paragraph Summary

The proposed research develops a multi-agent system that begins with a natural-language description of a stakeholder and task, automatically derives the architectural questions that must be answered, selects appropriate architecture views using a knowledge base, plans how to investigate the repository, executes that plan through LLM-based evidence extraction and grounding agents, reconciles desired information with available evidence, and generates validated task-oriented views with traceability, limitations, and confidence. The intended scientific contribution is not merely automated diagram generation, but an evidence-aware process for generating the right architecture view for the right stakeholder task.